Agentic Synthesis against Counterexample-Supplemented Sketches

Muness Castle Eric Rubeck

July 12, 2026

Abstract

Coding agents make plausible repairs cheap. Teams still need a durable way to carry domain corrections into future runs. A correction should leave more than a passing test or a chat note: it should name the rejected repair, the rule that rejects it, and the code or prompt surface that must preserve it.

We use the term *counterexample-supplemented sketch* for the repository artifact and *agentic synthesis* for the loop that edits against it. A human writes a partial program-like sketch. A finite corpus stores observations and promoted counterexamples. Subject-matter experts correct concrete outputs. A coding agent edits deterministic code and prompt surfaces under known-code anchors. A gate blocks completion until replay and semantic comparison pass for every promoted case.

The loop borrows from sketching and counterexample-guided inductive synthesis (CEGIS), but the execution surface is different. Classical sketching and CEGIS can make formal claims when the search space, specification, and checker are formal. Here, the synthesizer is an IDE-shaped coding agent whose edits span ordinary source code and LLM prompt surfaces. The claim stays bounded: when the gate passes, the current strategy satisfies the repository's encoded replay and comparison predicates for the promoted corpus.

The originating deployment ran inside a proprietary enterprise harness: a custom Cursor extension, `cursor://`-style commands, and an embedded web application where subject-matter experts loaded production examples, corrected outputs, and helped turn those corrections into input/output specifications. The public companion repository provides CatSynth, a synthetic browser application and a captured coding-agent experiment. The experiment gives the Developer one active failure at a time, lets it revise the sketch, deterministic code, and prompt together, and gates every revision against the full promoted corpus. A closed-world run separately gives the same model a complete immutable specification; it reaches 20/20 visible and 21/21 hidden cases in four Developer calls. In the open-world run, eight of fourteen proposed cases fail and are promoted. Retained Sketch–CE uses 217,576 Developer tokens, versus 400,081 for replaying all discovered cases and 371,050 for rebuilding from the evolved sketch. All three pass eight visible cases; their hidden scores are 15/21, 19/21, and 18/21 respectively. These are inspectable runs, not general performance claims.

1 Introduction

Coding agents make repairs cheap before the repository has captured every domain rule the repair must obey. A developer can ask for a patch from a failing test, a state delta, a prompt instruction, or a reviewer note. The agent can satisfy that signal and still violate the rule that made an SME reject the first repair. If the rule stays in chat or review memory, the next agent run can make the same plausible mistake.

The repair loop turns that rejected repair into a repository constraint. An SME correction supplies the approved output and the rule it exemplifies; a maintainer decides whether to promote it. The Developer then revises the current sketch, deterministic code, and prompt-mediated surface under known-code anchors. It receives one active failure. The gate receives the initial anchor and every promoted case. No new counterexample is revealed until the full gate is green.

Tests and prompts remain part of the loop. Tests catch shape errors, exceptions, bad deltas, and regressions. Prompts can state intent for the current run. A promoted counterexample has a different job: keep an SME correction available to later agents as an executable repository check.

The loop needs five artifacts:

1. a **sketch** $\mathcal{S}$: a partial program in prose and code-shaped structure;
2. a **counterexample-supplemented corpus** $\mathcal{E}$: examples, observations, and promoted failures;
3. **known-code anchors** $\mathcal{K}$: codebase-local shapes, APIs, conventions, and reference paths the agent must preserve;
4. a **dual-oracle implementation surface**: deterministic code for encodable holes and prompt-mediated completion for narrative holes;
5. a **gate** $\mathcal{G}$: replay plus semantic comparison over every promoted case.

Method at a glance

Reviewer correction promotes one failing case into $\mathcal{E}$

$\Downarrow$

Developer revises sketch $\mathcal{S}$, code, and prompt for that failure

$\Downarrow$

Gate runs the initial anchor and all of $\mathcal{E}$

$\Downarrow$

Green: reveal the next case; red: return one failed regression

The loop turns a tempting wrong repair into durable repository guidance.

The initial sketch records the human's current strategy: which repairs are permitted, prioritized, or forbidden. During synthesis, the Developer may reorganize or generalize that strategy while it edits code and prompts. The corpus records cases that have earned specification authority through promotion. Reviewer-approved outputs remain authoritative; a model-drafted explanation cannot overrule them. The gate is the executable boundary for the current done state.

Claim. A repository-native workflow is inspectable when every promoted counterexample connects four things: the sketch clause it supplements, the implementation or prompt surface that handles it, the replay check that verifies state repair, and the semantic compare check that rejects the tempting wrong repair. Passing the gate gives maintainers a bounded invariant over the current promoted corpus, current replay/checker code, and current golden rows. It does not claim correctness outside those artifacts.

Boundary. The claim is bounded. Existing agentic workflows can make tests and prompts durable. Classical CEGIS already has counterexample discipline when its formal assumptions hold. Repository work is smaller and messier: the human strategy, the promoted counterexample, and the executable check often live in separate artifacts. The repair loop stores the links among them without treating the result as a solver-backed proof.

What the paper provides.

1. a repository-native artifact: the counterexample-supplemented sketch;
2. a process model that turns subject-matter expert corrections into input/output specs, counterexamples, and golden data;
3. two hole-filling roles: Oracle A, deterministic code the agent maintains, and Oracle B, prompt-mediated completion for sketch-declared narrative holes;
4. a finite-corpus theorem for replay/compare gates and the proof obligation it leaves behind;
5. a worked example and companion repository for runnable inspection.

2 Originating setting

The enterprise deployment was the proving ground. Production failures arrived as concrete rows with incorrect outputs. SMEs loaded those rows into a review surface, corrected the outputs they expected, and chose which failures should become part of the golden set. Each promoted case then had to survive replay before another agent repair could pass.

That pressure appeared before the loop was formalized. CEGIS fit the need because it treats a failure as a case to keep and rerun. We adapted that discipline to agentic repair: a production failure became an input/output specification, the repository recorded the corresponding sketch change, an agent edited code or prompt surfaces, and the gate replayed the promoted scenario and compared policy-relevant fields.

The deployment itself included an SME review application, local agent actions from the IDE, model-backed code and prompt repair, and a harness for fixtures, replay, semantic compare, golden corpus promotion, and provenance. The AWS and IDE plumbing mattered operationally. The evidence boundary is narrower than that plumbing: SMEs judged concrete outputs; selected corrections were promoted; the harness replayed promoted cases; the golden set made regressions visible.

The public artifact uses synthetic data and public rules. It keeps the inspectable shape of the deployment: paper, fixtures, tests, provenance, and a small remediation case. A reader can inspect how a correction becomes a promoted counterexample, how the sketch records the rule, how an agent repair changes code or prompts, and what the replay/compare gate checks. The claim stays bounded to that inspectable shape.

3 Problem

Coding agents make plausible patches cheap. A patch can follow the prompt, match local style, pass the available tests, and still violate a domain rule that no one has written into the repository. An unencoded rule can make the patch wrong even when the agent followed the task.

The repair often goes this way:

1. A user gives an agent a domain task.
2. The prompt names the state gap.
3. The agent finds a local patch that closes that gap.
4. A subject-matter expert recognizes a policy violation.
5. The violation was present in the domain, yet absent from the checkable artifacts.

The patch can be technically clean. The agent can follow local style. The tests can pass. A promoted counterexample records the rule the patch violated: "this plausible repair is forbidden for this reason, and future repairs must satisfy this rule." If the team keeps that rule in conversation, the repository cannot reject the same repair again.

3.1 Tests need the rule they protect

A test checks behavior that has been encoded. A rule-linked fixture records the policy choice that makes one passing repair acceptable and another passing repair wrong. Two patches can produce the same expected output while taking different policy paths; the expected output alone hides that choice.

A promoted counterexample ties the failing fixture to the rule it exposed. Replay can show that the repaired row reaches the expected state. Semantic compare can check policy-bearing fields such as operation, target, date, and status. A sketch clause records why those fields matter, so a future maintainer or agent sees which policy ordering the assertion protects.

3.2 Prompts need repository anchoring

Prompts can carry domain intent before the team has reduced that intent to deterministic code. They can name business rules, narrative criteria, exception handling, and domain language. They are fragile when they live only in a chat transcript, notebook, model call, or agent session outside the repository's review path.

In a repository-native repair, prompt text is a versioned implementation surface. Oracle B is prompt-mediated completion constrained by the same sketch and promoted corpus as deterministic code. Prompt text may encode narrative criteria, but the produced rows still run through the same replay and semantic compare gate. The repository stores the prompt, the fixture, the sketch clause, and the check together.

3.3 A repository-native version of the loop

Formal synthesis and CEGIS cover cases where the template, specification, and checker fit inside a formal language. In that setting, a solver can search a constrained space and return a result with stronger formal guarantees than a repository gate. Counterexamples may come from a verifier, examples may guide inductive search, and oracles may steer component selection or specification refinement [17, 9].

In the repository case, SMEs can recognize a wrong repair before the team can fully formalize the rule it violated. The repository has to keep that rule tied to the artifacts that enforce it: sketch clause, promoted counterexample, prompt or code surface, replay check, semantic compare fields, and generated provenance. Some rules move into deterministic code. Others remain in prompts while the team gathers enough promoted counterexamples to encode them more tightly.

The gate makes a bounded claim over the current sketch, promoted corpus, implementation surface, replay predicate, and semantic compare predicate. That claim prevents the last plausible repair from disappearing back into a chat transcript, test failure, or reviewer memory.

4 Lineage

4.1 Sketch and CEGIS

Sketching supplies the discipline: the programmer supplies structure and the synthesizer searches within it [15, 16]. Earlier sketching work already used counterexamples to refine candidates in finite-program synthesis settings [17]. CEGIS adds the loop: generate a candidate, validate it, and feed counterexamples into the next step. Oracle-guided synthesis adds an oracle that guides or validates component choices [9].

The boundary differs in an ordinary repository. The agent searches across code, prompts, fixtures, and checks. A promoted counterexample records the rule that the repair must preserve and the known bad repair that the gate must reject.

4.2 Programming by example

Programming-by-example systems such as FlashMeta and PROSE synthesize DSL programs from examples using domain-specific deduction [13, 8]. The borrowed discipline is that examples can constrain synthesis when the language and operators fit the task. Here the example set serves a repository repair loop. Each promoted case ties an SME correction to a sketch clause, an implementation or prompt surface, and a replay/compare check.

4.3 Agentic coding and tests

Agent benchmarks such as SWE-bench evaluate whether language models can resolve real repository issues [10]. Tests-as-prompts work studies tests as both input and evaluation for LLM code generation [5]. The borrowed discipline is to steer agents with repository artifacts and executable checks.

The boundary is the unit of review. For policy-heavy repairs, every promoted counterexample should name the sketch clause that changed, the tempting repair it rejects, the code or prompt surface that handles the rule, and the gate check that enforces it.

4.4 Prompt engineering, specs, and natural-language coding

Prompt programming and prompt-based learning treat natural-language prompts as a control surface for language models [14, 12]. Language-Oriented Programming argues that natural language can lower the barrier for non-experts who contribute to software projects [3]. Promptware engineering asks teams to treat prompts like software artifacts with requirements, tests, debugging, evolution, and monitoring [4]. Copilot Workspace describes a product path from idea to code to software in natural language [6]. End-user software engineering is the older concern: domain experts often create or maintain computational artifacts without being professional software developers [11].

Spec-driven agent tools make a related move. Kiro uses requirements, design, and tasks to make model assumptions visible before implementation [18]. GitHub Spec Kit centers agentic development on a Spec–Plan–Tasks–Implement sequence and treats specs as structured context for agents [7]. Specification by Example supplies the older discipline: concrete examples become living documentation when they are executable enough to constrain future work [1].

Sketch-CE makes the destination of counterexample feedback explicit. A failure can lead the Developer to repair code or prompts under the current sketch, leaving the governing rule unchanged. It can also expose a missing or mistaken rule, leading a maintainer to approve a sketch revision and require the implementation to satisfy the revised rule. In Argyris's terms, the first path is a single-loop correction; the second is double-loop learning because the correction reaches the governing policy or objective [2]. The gate preserves the executable result of either path across later repairs, while the versioned sketch records the revised rule.

Living-specification workflows can support the same kind of learning. The narrower distinction is procedural: Sketch-CE places reviewed sketch revision inside the counterexample repair protocol instead of treating it as an exceptional upstream process. The Developer may propose a revision, but the SME or maintainer decides whether the counterexample is valid, which rule it exposes, whether the sketch should change, which expected result is authoritative, and what the gate must enforce. If the Developer rewrites the sketch merely to justify its patch, the protocol has failed.

The boundary is that natural language is one mutable surface among several. In this loop, a spec means the sketch, promoted counterexamples, and replay/compare harness together. If a rule lives in a prompt, the prompt gets the same review pressure as code: a sketch clause, promoted examples, replay, semantic compare, and an owner.

4.5 Positioning boundary

The repository loop uses those disciplines at a narrower boundary. The sketch records the rule. The corpus records SME corrections and counterexamples. The coding agent edits code and prompts. The gate replays the case and compares the semantic fields that define success.

	Sketch / CEGIS	Programming by example	Repository loop
Human input	Partial program with holes	Examples inside a DSL	Sketch, SME corrections, counterexamples, anchors
Synthesizer	Solver-backed search	DSL learner	Coding agent editing code and prompts
Failure signal	Counterexample from validator	Missing or inconsistent example	Gate failure or SME correction
Validation	Formal or bounded decision procedure	Example consistency	Replay plus semantic compare over $\mathcal{E}$
Result boundary	Solver-relative result	DSL/example consistency	Finite-corpus predicate satisfaction under repo semantics

The last column is the checklist for a promoted case: sketch clause, counterexample, edited code or prompt surface, and gate check that rejects the known bad repair.

5 Artifacts

For each promoted case, readers should be able to find five repository artifacts. The sketch records the rule. The corpus stores the promoted cases. Known-code anchors record local conventions for the agent. The implementation surface is the code or prompt text the agent edits. The gate checks the result against replay and semantic comparison. When a gate fails, maintainers can ask which artifact needs work: the sketch, corpus, anchor, implementation surface, or checker.

Definition 1 (Sketch)**.** *A sketch $\mathcal{S}$ is a reviewable, agent-editable artifact that records the strategy space the agent may use and the holes it may fill. It may contain prose, tables, pseudo-code, type shapes, policy order, abstention rules, and examples of forbidden repairs. A human supplies the initial sketch. During synthesis the Developer revises it with code and prompts; maintainers review the result.*

Artifact	Reader question	Failure if blurred
Sketch	What rule should future repairs preserve?	Agents repair from prompt context and the rule stays hidden.
Corpus	Which cases have specification authority?	Maintainers treat examples as anecdotes, not promoted constraints.
Anchors	What local code shape must the agent preserve?	The agent invents a second convention.
Implementation surface	Which code or prompt applies the rule?	A paper rule cannot be traced to the surface that handles it.
Gate	What behavior is executable now?	Readers cannot see what the repository checked.

Table 1: Artifact responsibilities from a reader's point of view.

Maintainers and agents use the sketch to find the policy shape:

1. Which operations exist?
2. Which operation has priority when several repairs close the same state gap?
3. Which fields are policy-bearing?
4. Which decisions belong in deterministic code?
5. Which decisions belong in prompt-mediated narrative completion?
6. Which cases force abstention or escalation?

The sketch is less formal than a full specification. Maintainers use it to hold rules that are known but not yet fully encoded. They can later move a rule into deterministic code, prompt text, fixture data, or human review without losing its repository home.

Definition 2 (Corpus)**.** *The corpus $\mathcal{E} = \{(x_i, y_i)\}_{i=1}^{n}$ is the finite set of promoted examples the current gate must satisfy. Each input x_i is a concrete scenario or fixture. Each expected output y_i is the semantic repair the gate enforces.*

Maintainers give the corpus specification authority by promotion. Before promotion, an example can be noisy and an SME correction can be ambiguous. For each promoted counterexample, maintainers should record a plausible wrong output, state the rule it violates, and encode the check in the gate. After that review, maintainers treat observed behavior as specification pressure.

Definition 3 (Known-code anchor)**.** *A known-code anchor $\mathcal{K}$ is an existing source artifact the agent must follow while editing: an API, result type, parser convention, error shape, fixture format, test helper, reference implementation, or prompt structure. Maintainers use anchors to reduce the risk of parallel local inventions.*

Maintainers use anchors to tie a rule to the repository shape that already carries similar rules. The sketch records the policy. Anchors record the local conventions the agent should preserve while editing. They matter when a local test can pass with a new result shape that no caller expects.

Definition 4 (Implementation surface)**.** *An implementation surface is the versioned code or prompt text that applies a sketch rule. Deterministic code surfaces handle rules that can be encoded directly. Prompt-mediated surfaces handle narrative or policy choices that still require language-model completion.*

Agents repair implementation surfaces under the sketch, corpus, and anchors. Readers should be able to trace the surface from the promoted case that exposed the hole to the gate check that rejects the known wrong repair. If that trace is missing, a rule can remain true in prose while no edited code or prompt applies it.

Definition 5 (Gate)**.** *A gate $\mathcal{G}$ is the executable validation bundle that evaluates a candidate strategy $\mathcal{H}$ over every promoted case in $\mathcal{E}$. The gate has two layers: replay and semantic compare.*

Replay checks whether the proposed output changes the world state according to the encoded state predicate. Semantic compare checks whether the output used the encoded policy-bearing fields from the promoted expectation. Reporting the two layers separately lets maintainers distinguish state repair from policy repair. That distinction matters because the most dangerous repair can be the one that makes the arithmetic look right while choosing the wrong operation.

6 Roles

The loop works only if each actor has a narrow job. SMEs judge concrete outputs and explain corrections. Maintainers decide which corrections become specification and keep the machinery and anchors inspectable. Agents repair implementation surfaces under those constraints. Gates rerun and compare promoted cases; they do not decide domain truth.

6.1 Subject-matter expert

The SME judges concrete outputs through a review interface. When an output is wrong, the SME supplies the corrected output and the reason it is correct. The review action gives the maintainer evidence to promote a case. If the harness extracts an input/output spec, that extraction is clerical; the SME's correction is the source of domain judgment.

The SME's correction has three parts:

1. the observed input;
2. the corrected output;
3. the explanation that distinguishes the corrected output from the tempting wrong one.

The explanation matters because it guides the sketch update and tells the agent what general rule the fixture exemplifies. A corrected row alone can invite memorization. A corrected row plus a reason says which tempting repair the loop must reject.

6.2 Platform maintainer

The maintainer owns promotion and the machinery around it. They decide when an SME correction enters $\mathcal{E}$, keep the sketch and fixtures tied to the promoted case, and maintain the runtime, review path, replay, semantic compare, tests, and provenance.

SMEs supply the judgments. The maintainer promotes those judgments into repository artifacts and keeps promotion repeatable: the runtime runs, the gate checks the right fields, the output shape stays compatible, and promoted rules remain inspectable.

6.3 Coding agent

The agent repairs the artifact under constraints. It may edit deterministic code, prompt text, fixtures, or tests. It does not decide which domain rule is true. Its job is to make a repair that satisfies $\mathcal{S}$, $\mathcal{E}$, $\mathcal{K}$, and $\mathcal{G}$ without breaking the repository's existing shape.

The agent should receive four forms of context before editing:

1. the current sketch, code, and prompt;
2. one active counterexample or failed regression, projected to its checked fields;
3. the reviewer policy that explains the failure;
4. the known-code anchors that define output shape and local conventions.

The full promoted corpus is not repair context. It belongs to the gate. With one active failure, the agent has a narrower job: generalize the rule, reject the known tempting patch, and keep the codebase's existing shape.

6.4 Oracle A: deterministic code

Oracle A handles policy that maintainers can encode as deterministic, reviewable source: grouping, filtering, ordering, type construction, deterministic abstention, and domain calculations. The agent can repair Oracle A because the rule has been made concrete enough to encode. The promoted SME correction remains the source of truth.

6.5 Oracle B: prompt-mediated completion

Oracle B handles narrative or under-encoded policy: ambiguous notes, human descriptions, client-specific language, or judgment that belongs in a model prompt at the current stage of the system. The prompt has no authority on its own. Oracle B remains constrained by the sketch and checked against the SME-approved corpus by replay and semantic compare.

6.6 Hybrid resolver

A hybrid resolver tries Oracle A first, then routes abstentions to Oracle B. Maintainers keep deterministic policy on the reliable path and leave narrative completion explicit. A rule can begin in prompt-mediated completion, then move into deterministic code once enough counterexamples make it precise.

7 Counterexample lifecycle

The sequence begins with a partial sketch, not with a finished implementation. The Developer first generates the sketch, deterministic code, and prompt surface. An initial acceptance case must pass before the first domain counterexample is revealed. Thereafter, the Developer sees one active failure while the gate sees the entire promoted corpus.

Repository synthesis loop

```
(S, code, prompt) = Developer(S0, empty code, empty prompt, K)
while Gate(initial anchor) fails:
    (S, code, prompt) = Developer(S, code, prompt, one failure, K)

for proposed reviewer case c:
    if c passes current implementation:
        record c as coverage; do not promote it
        continue
    promote c into E
    active = c
    repeat:
        (S, code, prompt) = Developer(S, code, prompt, active, K)
        failures = Gate(initial anchor + all of E)
        active = one failed regression
    until failures is empty
```

A passing proposal does not become a counterexample merely because it was scheduled for review. It is coverage. A failing proposal becomes specification only after review and promotion. A new proposal remains hidden until all current regressions pass.

7.1 Observation

An observed case reaches the review surface. It can come from production data, a synthetic fixture, a regression report, or a reviewer. The system proposes an output. The SME marks the output correct or supplies a correction. At this point the case is evidence; no one has promoted it as specification.

7.2 Correction

The SME correction names the desired output and the rule behind it. The reviewed expected output has specification authority; an LLM may draft a generalized clause or explanation but may not replace the approved result. When useful, the SME also names the tempting wrong repair: an output that could satisfy replay while violating policy. That gives the maintainer a rule to inspect instead of a bare bad row.

7.3 Spec extraction

The harness or maintainer translates the correction into an input/output spec. An LLM can draft the general sketch clause, especially when the SME explanation is prose. The maintainer reviews that draft. The model's proposed wording is advisory; the reviewed expected fields remain the source of truth.

7.4 Promotion

The maintainer promotes only cases they are willing to treat as specification. They add the case to $\mathcal{E}$ with enough information to reject the tempting wrong repair. They treat promotion as a deliberate specification change, not automatic elevation of every correction.

Before promotion, the harness runs the proposed case against the current implementation. If the case already passes on the fields it claims to constrain, the harness records coverage and does

not add it to $\mathcal{E}$. During promotion, the maintainer also checks the attached artifacts. The comparer may need a new field, the fixture may need a clearer name, and the failure packet must expose only the fields this counterexample constrains.

7.5 Sketch revision

The Developer revises the sketch together with code and prompts. It may generalize the active failure, reorganize earlier clauses, or leave a newly declared policy hole open. It receives the current sketch and implementation, known-code anchors, and one projected failure packet. It does not receive the unrevealed cases or the full promoted corpus.

The revised sketch should state general policy rather than paste the concrete row. Reviewers can reject a sketch edit that invents policy or erases an earlier rule, while the full gate catches behavioral regressions already represented in $\mathcal{E}$.

7.6 Repair

The Developer repairs Oracle A, Oracle B, and the sketch under known-code anchors. A repair turn targets exactly one failure. It can change both deterministic and prompt-mediated surfaces when the counterexample crosses that boundary. The gate, rather than the repair prompt, carries the responsibility for all earlier promoted behavior.

7.7 Gate

The gate runs replay and semantic compare over the initial anchor and all promoted cases. If a prior case fails, the harness selects one failed regression as the next active Developer input and reruns the full gate after the revision. A passing gate supports the Section 9 result for the current strategy, corpus, checkers, fixtures, and golden rows. If any of those change, the maintainer must run the gate again.

8 Gate semantics

The gate runs the checks tied to the current promoted corpus. It does not prove the program correct. It answers a narrower question: for every promoted row, does the candidate repair the encoded state and preserve the encoded policy fields?

Definition 6 (Replay)**.** *Replay applies a candidate output r to an input state x and checks the encoded state repair. It accepts when the proposed change closes that state gap under the domain-specific replay code. It rejects when the state gap remains or the candidate cannot be applied. Replay may compute balances, apply updates, recalculate derived fields, or simulate downstream effects.*

Definition 7 (Semantic compare)**.** *Semantic compare checks policy-bearing fields against the promoted expectation y: operation kind, selected entity, work date, issue type, status mutation, route, priority, or any other field encoded by the comparer. It accepts when those fields match. It rejects a mismatch even when replay accepts the state repair.*

Definition 8 (Gate pass)**.** *For a strategy $\mathcal{H}$ and corpus $\mathcal{E}$, the gate passes iff every $(x_i, y_i) \in \mathcal{E}$ yields $r_i = \mathcal{H}(x_i)$ such that replay accepts (x_i, r_i) and semantic compare accepts (y_i, r_i).*

Replay and compare must stay separate because they catch different wrong repairs. A candidate can repair state and still choose the wrong policy field. For example, it can close the hours math

Candidate behavior	Replay	Compare	Interpretation
Open state gap	reject	not reached	Candidate fails encoded state repair.
Closed state, wrong op	accept	reject	Candidate repairs state but violates a policy field.
Closed state, right op	accept	accept	Case passes the current gate checks.

Table 2: Replay and semantic compare reject different failures.

while selecting append instead of update; replay accepts the state change, and compare rejects the operation kind.

8.1 Design rules for gates

A gate should be deterministic, local, and specific. It should run locally, identify the promoted case that failed, and preserve the difference between replay failure and compare failure. The failure message should point the repair agent to the rule to inspect.

Good gates have these properties:

1. **Promoted corpus coverage.** Every promoted case runs.
2. **Failure specificity.** Replay and compare failures are reported separately.
3. **Deterministic CI path.** Gate-critical checks do not depend on a live model call.
4. **Source links.** Each failure can be traced to sketch clause, scenario, and check.
5. **Tempting-patch rejection.** The gate includes at least one check that fails the previously plausible wrong repair.

These rules are design constraints, not formal assurances. A gate that omits a field cannot check that field. A comparer with a bug can accept the wrong behavior. Any correctness claim is bounded by the current corpus and the current checkers.

9 Finite-corpus correctness

When $\mathcal{G}$ passes, it proves only the cases in $\mathcal{E}$ under the repository's current replay function, semantic comparer, fixtures, and golden rows. It does not prove behavior on cases outside $\mathcal{E}$ or on policy fields the checker does not encode. When maintainers promote a new failure into $\mathcal{E}$, the gate must check one more case. After it passes again, the theorem covers the expanded corpus.

Definition 9 ($\mathcal{E}$-correctness)**.** *A strategy $\mathcal{H}$ is $\mathcal{E}$-correct with respect to replay function R and compare predicate C when, for every $(x_i, y_i) \in \mathcal{E}$, $R(x_i, \mathcal{H}(x_i)) = \text{true}$ and $C(y_i, \mathcal{H}(x_i)) = \text{true}$.*

Lemma 1 (Replay soundness relative to the implementation)**.** *If replay returns true for (x, r), then r satisfies the state-repair condition encoded by replay for x.*

Proof. For this gate, replay is the executable state-repair predicate. It evaluates the candidate output against the input state and returns true exactly when the encoded state-repair predicate holds. A true replay result entails the state-repair condition the gate implements. □

Lemma 2 (Compare soundness relative to the corpus)**.** *If semantic compare returns true for expected output y and candidate output r, then r agrees with y on every policy-bearing field encoded by the comparer.*

Proof. The comparer enumerates the checked policy-bearing fields and fails on mismatch. A true compare result means every encoded policy-bearing field matched the promoted expectation. □

Theorem 1 (Finite-corpus gate soundness)**.** *If $\mathcal{G}$ passes for strategy $\mathcal{H}$ on corpus $\mathcal{E}$, then $\mathcal{H}$ is $\mathcal{E}$-correct with respect to the replay and compare semantics encoded by the repository.*

Proof. By definition, $\mathcal{G}$ iterates over every $(x_i, y_i) \in \mathcal{E}$, computes $r_i = \mathcal{H}(x_i)$, and requires replay and compare to pass. By replay soundness, each accepted r_i satisfies the state-repair condition encoded by replay for x_i. By compare soundness, each accepted r_i agrees with y_i on every encoded policy-bearing field. Thus every promoted corpus case satisfies both predicates, which is exactly $\mathcal{E}$-correctness. □

Scope. The theorem is bounded to the promoted corpus, the encoded replay checker, the encoded semantic comparer, and the current golden rows. It does not cover unpromoted failures, unencoded policy fields, future model behavior, or private deployment behavior outside the public artifact. If a checker is wrong, maintainers must fix the checker. If a golden row is wrong, an SME must correct it. A maintainer can audit the boundary: which predicates passed, which corpus they covered, and which questions remain outside the proof.

10 Worked example at sketch level

10.1 Scenario

CatSynth presents a synthetic owner profile with one visible preference gap and one hard policy rule. The owner wants a large, fluffy, affectionate cat and has an allergy trait. The naive resolver selects Persian because it maximizes the encoded preferences. The policy resolver selects Siberian because the fixture marks it as satisfying the same preferences and the hard allergy rule. These are illustrative fixture attributes, not pet-selection or medical advice.

The ambiguity is the case's value. Both candidates close the state gap defined by the replay predicate. Only one satisfies the policy-bearing fields in the promoted expectation. The counterexample preserves the near miss so a later agent cannot recover the tempting preference ranking and still pass the gate.

Repair	State effect	Policy meaning	Gate result
Persian	preferences satisfied	hard rule ignored	replay accepts; compare rejects
Siberian	preferences satisfied	hard rule respected	replay accepts; compare accepts

The worked example separates state repair from policy repair in one small case.

10.2 Tempting repair

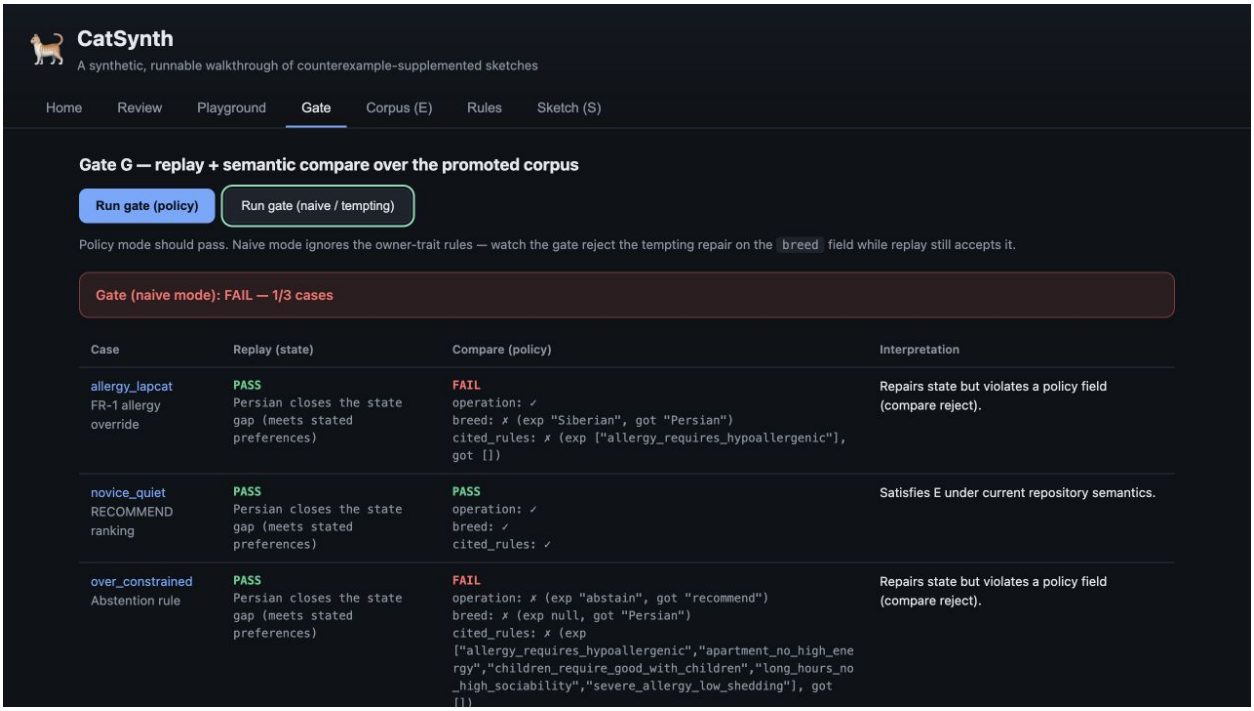
The tempting repair ranks breeds only by preference match. Replay accepts Persian because the fixture marks it as large, fluffy, and affectionate. Semantic compare rejects the output because the promoted expectation names Siberian and cites the hard allergy rule. The state gap is closed by an output that violates the encoded policy.

10.3 Sketch rule

The promoted counterexample adds a sketch rule: preference match never overrides a hard rule. The sketch tells the next agent that filtering precedes ranking. It also names the fields that future changes must preserve: operation, selected breed, and cited rule identifiers.

10.4 Replay and compare

Replay asks, "did the proposed output repair the encoded state gap?" Compare asks, "did it use the encoded rule the sketch requires?" CatSynth needs both questions because the naive preference result passes the first check and fails the second. The promoted case guards the policy choice as well as the visible preference match.



CatSynth makes the two predicates visible: the naive strategy repairs the encoded preference gap, so replay passes, but it does not preserve the promoted breed and cited-rule fields, so semantic compare fails.

The illustrated companion at `paper/catsynth-worked-example.md` follows the same case from the sketch and anchors through the tempting output, promoted corpus row, and gate result.

10.5 Captured synthesis trajectory

The browser shows the finished artifacts. The experiment reconstructs their synthesis. It starts with the initial sketch and empty deterministic and prompt files. GPT-5.4-mini at low effort acts as Developer; tools and environment access are disabled. Each successful revision is copied into a separate generation directory with the exact active failure and full gate.

Generation	Newly checked policy	Before repair	Gate
000	Initial preference strategy	Empty implementation	1/1
001	Allergy hard filter	Persian, no cited rule	2/2
002	Hard-rule composition and abstention	Balinese, one cited rule	3/3
003	Travel note maps to `avoid_needy`	No Oracle tag	4/4
004	Tag penalty and base-score boundary	Balinese with correct tag	5/5
005	Distinct soft rules compose	Incomplete soft ranking	6/6
006	Duplicate soft concern applies once	Missing narrative tag	7/7
007	Unknown allergy status escalates	Unsafe recommendation	8/8
008	Malformed applicable policy escalates	Ignored policy error	9/9

The separation between generations 003 and 004 is deliberate. Generation 003 checks only the prompt-mediated tag. Its failure packet contains no breed values and leaves the deterministic meaning of the tag open. The next proposed case then fails because the correct tag does not yet produce the approved ranking. That independent failure becomes the fourth counterexample.

The main decision is not "iteration or one shot." It is whether the governing policy is already complete. In a separate closed-world run, the model receives an immutable complete specification and empty implementation files. It reaches 20/20 visible and 21/21 hidden cases after four Developer calls and 132,632 Developer tokens. When that premise is true, spec-first is the simpler approach.

The open-world run adds two controls to isolate retained state. Replay-all rebuilds from the initial sketch and all promoted cases known at each epoch. Evolved-sketch rebuild receives the current sketch but never the full counterexample corpus; after a failed gate it receives the visible failure packets. Retained Sketch–CE uses 9 Developer calls and 217,576 Developer tokens, replay-all uses 15 and 400,081, and evolved-sketch rebuild uses 16 and 371,050. Their hidden scores are 18/21, 15/21, and 19/21. Sketch–CE reduces implementation work and churn in this run, but does not achieve the best hidden score or the lowest total model-token count.

11 Applying the method

Start this loop in an ordinary repository. Do not wait for a harness. Use the files the team already trusts: design notes, fixtures, parser tests, golden rows, prompt templates, or review checklists. For each agent repair, record three answers in the repository: which rule governs the repair, which example exposed the rule, and which gate would reject the tempting wrong patch.

11.1 Start with the sketch

Write the known strategy before asking the agent for code. Put the first sketch in a short file that names the allowed operations, priority order, known holes, and abstention cases. Give later failures a clause to attach to. Do not present the first sketch as a complete policy. Ask the Developer to return the revised sketch with every implementation generation, and review that artifact with the code and prompt it describes.

11.2 Collect discriminating examples

Keep examples that separate two plausible repairs. Use a happy-path case to show the intended output. Use a counterexample to show a repair that looks green but violates the rule. When an SME corrects an agent, save the case if it teaches that boundary. Do not promote examples only because they are available; promote the ones that would catch a wrong future edit.

11.3 Name known-code anchors

Point the agent at the repository facts it should reuse: result types, parser behavior, fixture format, error shape, prompt conventions, and naming rules. Name the exact files, symbols, or clauses when you can. Use those anchors to keep the agent from inventing a second local convention just to pass the current case.

11.4 Separate replay from compare

Build replay and compare as separate checks. Use replay to check whether the repaired state now satisfies the predicate that failed. Use compare to check whether policy-bearing fields still match the promoted expectation. Do not hide both jobs inside a single assertion that says "the output looks right." Read the failure by its job: replay failure means the state is still broken; compare failure means the state repair passed while a policy field diverged.

11.5 Promote failures deliberately

Triage each proposed case before adding it to $\mathcal{E}$. Run it first. A passing proposal is coverage, not a counterexample. Treat fixture bugs as fixture work and checker bugs as checker work. Promote only failures that expose missing policy, then give the Developer that one failure so it can revise $\mathcal{S}$ and the implementation together.

For each promoted case, record four things: the tempting patch it rejects, the sketch clause that changed, the replay or compare predicate that checks it, and the failure message that should guide a future repair. This work turns one corrected answer into reusable specification pressure.

12 Companion artifact

The companion repository is the public inspection path for the paper's bounded claim. It publishes CatSynth's initial sketch, proposed cases, Developer and Oracle adapters, browser application, gates, tests, and full generated histories. The executable `experiment/run_experiment.py` implements the Sketch–CE and closed-world spec-first loops. `experiment/adaptive_open_world_experiment.py` freezes a candidate pool, reveals one case at a time, archives every Developer generation, and runs the two open-world controls. The illustrated supplement traces those artifacts through the captured run, interface, and source files.

Readers can inspect the repository to answer four concrete questions:

1. Which single failure was visible to each Developer generation?
2. How did the sketch, deterministic code, and prompt change together?
3. Which full-corpus gate ran after each revision?
4. Which proposed cases were promoted as failures or retained only as coverage?

That inspection matters because the claim is finite and executable. A passing gate says that the current artifact satisfies the encoded replay and comparison predicates for the promoted corpus. Production data, proprietary workflow code, private SME decisions, and behavior outside the public corpus remain outside the artifact.

13 Discussion

13.1 Tests name behavior; sketches name intent

A passing test suite covers only encoded behavior. Tests check behavior in the current run; sketch clauses preserve why a failure mattered for the next repair. When a case enters $\mathcal{E}$, record the rule a future patch must preserve alongside the input and expected output.

13.2 Prompts implement one surface of the contract

Treat prompts as code that implements policy. If a rule lives in instructions or narrative notes, bind that surface to the same sketch and corpus as source code. Review prompt edits as contract changes, and judge their outputs at the gate instead of treating them as informal wording.

13.3 SME corrections become specification inputs

Treat SME review as specification input. When an SME corrects an output, ask for the rule the output violated and decide whether that rule belongs in $\mathcal{E}$. A correction becomes spec only after a maintainer turns it into a promoted counterexample or sketch change; until then it is feedback.

13.4 Promotion quality is a scaling constraint

Promotion quality controls scale. Add a case when it rules out a plausible wrong repair. Leave out cases that only restate an accepted rule, duplicate an existing boundary, or lack a domain rule someone can name.

14 Limitations

1. **Finite corpus.** A passing gate proves only the current promoted $\mathcal{E}$. When a new failure appears, someone must promote it and run the gate again before the bounded result covers it. The result does not extend to unseen cases.

2. **Checker quality.** Replay and compare are only as good as the semantics encoded in their checkers. A buggy checker can certify the wrong behavior until a maintainer repairs the checker and replays the promoted corpus.

3. **Golden dataset quality.** A wrong row in the golden dataset makes the gate enforce the wrong behavior. A domain reviewer must distinguish a corrected answer from a corrected specification before the row becomes part of the claim.

4. **Promotion quality.** A reviewer should promote a case only when it states the domain rule and rules out a plausible wrong repair. Examples that only record an answer should stay out of the promoted corpus.

5. **Sketch quality.** Rules left outside the sketch still rely on human review. Maintainers must keep the sketch current as new counterexamples arrive.

6. **Prompt drift.** Live model behavior can drift. Review live output before promotion; do not treat a prompt response as a proof artifact.

7. **Single captured comparisons.** CatSynth records one stochastic run per path with one model and candidate order. The closed-world run begins with information the open-world run must discover, so their token totals answer different questions. Within the open-world experiment, the rebuild controls inherit the promoted stream without paying its discovery cost. The observed token ratios and checked results are not a benchmark or a causal estimate.

8. **Enterprise evidence boundary.** Readers can inspect the control structure in the public companion's sanitized fixtures. Public evidence covers that control structure, not private deployment behavior.

These boundaries mark where the proof stops. The gate checks the finite corpus with current replay, compare, and data; people still own promotion, checker design, golden review, sketch maintenance, prompt review, and any private deployment claim.

15 Conclusion

Do not trust an agent repair just because the patch looks plausible. Trust it only after an SME has stated the rule and the repository can rerun that rule against the repair.

For a promoted case, the required trail is: the SME correction, the one projected failure given to the Developer, the revised sketch and implementation, the full promoted corpus run by the gate, the replay predicate that checks state repair, and the semantic compare predicate that checks policy fields.

When those pieces are present, the agent has a bounded job: revise the sketch, code, and prompt for one active failure. The gate, not the repair prompt, carries the full promoted corpus. A passing gate does not prove the domain correct. It says only this: for the current repository, current checkers, and current promoted corpus, replay and semantic comparison pass. Everything outside those encoded cases and predicates still belongs to human review and future promotion.

References

[1] Gojko Adzic. *Specification by Example: How Successful Teams Deliver the Right Software*. Manning Publications, 2011.

[2] Chris Argyris. Double loop learning in organizations. *Harvard Business Review*, 55(5):115–125, 1977.

[3] Amin Beheshti. Natural language-oriented programming (NLOP): Towards democratizing software creation. In *2024 IEEE International Conference on Software Services Engineering*, SSE 2024, pages 258–267. IEEE, 2024.

[4] Zhenpeng Chen, Chong Wang, Weisong Sun, Xuanzhe Liu, Jie M. Zhang, and Yang Liu. Promptware engineering: Software engineering for prompt-enabled systems, 2025.

[5] Yi Cui. Tests as prompt: A test-driven-development benchmark for llm code generation, 2025.

[6] Thomas Dohmke. Github copilot workspace: Welcome to the copilot-native developer environment. GitHub Blog, 2024. Published April 29, 2024.

[7] GitHub. What is spec-driven development? GitHub Spec Kit Documentation, 2026. Accessed July 1, 2026.

[8] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2):1–119, 2017.

[9] Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, ICSE 2010, pages 215–224, 2010.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[11] Andrew J. Ko, Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin Erwig, Chris Scaffidi, Joseph Lawrance, Henry Lieberman, Brad Myers, Mary Beth Rosson, Gregg Rothermel, Mary Shaw, and Susan Wiedenbeck. The state of the art in end-user software engineering. *ACM Computing Surveys*, 43(3), 2011.

[12] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.

[13] Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, pages 107–126. ACM, 2015.

[14] Laria Reynolds and Kyle McDonell. Prompt programming for large language models: Beyond the few-shot paradigm. *arXiv preprint arXiv:2102.07350*, 2021.

[15] Armando Solar Lezama. *Program Synthesis by Sketching*. PhD thesis, EECS Department, University of California, Berkeley, December 2008.

[16] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15(5–6):475–495, 2013.

[17] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit A. Seshia, and Vijay A. Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS 2006, pages 404–415. ACM Press, 2006.

[18] Nikhil Swaminathan and Deepak Singh. Introducing kiro. Kiro Blog, 2025. Published July 14, 2025.